8. IMA(Integrity Measurement Architecture) 통합 분석

8.1 IMA 개요 및 파일 무결성 측정 원리

8.1.1 IMA 기본 개념

IMA(Integrity Measurement Architecture)는 Linux 커널에 내장된 파일 무결성 측정 시스템으로, 시스템에서 실행되는 파일들의 무결성을 실시간으로 모니터링합니다. Keylime은 IMA와 통합되어 원격 시스템의 런타임 무결성을 지속적으로 검증할 수 있습니다.

8.1.2 IMA 측정 프로세스

IMA는 다음과 같은 방식으로 파일 무결성을 측정합니다:

1. **파일 접근 시점 측정**: 파일이 실행되거나 mmap()으로 메모리에 매핑될 때 해시값 계산
2. **PCR 확장**: 계산된 해시값을 TPM의 PCR 10번 레지스터에 확장 저장
3. **측정 로그 기록**: `/sys/kernel/security/ima/ascii_runtime_measurements` 에 측정 기록 저장
4. **정책 기반 검증**: 런타임 정책에 따라 허용된 파일인지 검증

8.1.3 IMA 템플릿 형식

IMA는 여러 템플릿 형식을 지원합니다:

- **ima**: 레거시 형식 (SHA1 해시 + 파일 경로)
- **ima-ng**: 새로운 형식 (알고리즘 명시 + 해시 + 파일 경로)
- **ima-sig**: 디지털 서명 포함 형식

- **ima-buf**: 버퍼 데이터 측정 형식

8.2 keylime/ima/ima.py 런타임 정책 구조 분석

8.2.1 EMPTY_RUNTIME_POLICY 구조

`keylime/ima/ima.py` 에서 정의된 기본 빈 런타임 정책:

```
EMPTY_RUNTIME_POLICY: RuntimePolicyType = {
"meta": {
"version": RUNTIME_POLICY_CURRENT_VERSION, # 현재 버전: 1
"generator": RUNTIME_POLICY_GENERATOR.EmptyAllowList, # 생성기 타입
},
"release": 0, # 정책 릴리스 번호
"digests": {}, # 허용된 파일 해시 목록
"excludes": [], # 검증에서 제외할 파일 패턴
"keyrings": {}, # IMA 키링 정보
"ima": {
"ignored_keyrings": [], # 무시할 키링 목록
"log_hash_alg": "sha1", # IMA 로그 해시 알고리즘
"dm_policy": None # 디바이스 매퍼 정책
},
"ima-buf": {}, # IMA 버퍼 측정 정책
"verification-keys": ""  # 검증용 공개키
```

```
}
```

8.2.2 RuntimePolicyType 스키마

런타임 정책은 JSON 스키마로 검증됩니다:

```
RUNTIME_POLICY_SCHEMA = {
"type": "object",
"required": ["meta", "release", "digests", "excludes", "keyrings",
"ima", "ima-buf", "verification-keys"],
"properties": {
"meta": {
"type": "object",
"required": ["version", "generator"],
"properties": {
"version": {"type": "integer", "minimum": 0},
"generator": {"type": "integer"},
"timestamp": {"type": "string"}
}
},
"digests": {"$ref": "#/definitions/digests-object"},
"excludes": {"type": "array", "items": {"type": "string"}},
# ... 추가 속성들
}
```

```
}
```

8.2.3 정책 생성기 타입

```
class RUNTIME_POLICY_GENERATOR(enum.IntEnum):
Unknown = 0
EmptyAllowList = 1  # 빈 허용 목록
CompatibleAllowList = 2  # 호환 가능한 허용 목록
LegacyAllowList = 3  # 레거시 허용 목록
```

8.3 IMA 측정 목록 처리 및 검증

8.3.1 측정 목록 처리 함수

`process_measurement_list()` 함수는 IMA 측정 로그를 처리하는 핵심 함수입니다:

```
def  process_measurement_list(
agentAttestState: Optional[AgentAttestState],
lines: List[str], # IMA 측정 로그 라인들
runtime_policy: Optional[RuntimePolicyType], # 런타임 정책
pcrval: Optional[str], # 예상 PCR 값
ima_keyrings: Optional[ImaKeyrings], # IMA 키링
boot_aggregates: Optional[Dict[str, List[str]]], # 부트 집계값
hash_alg: algorithms.Hash = algorithms.Hash.SHA1 # 해시 알고리즘
```

```
) -> Tuple[str, Failure]:
```

8.3.2 검증 프로세스

1. **PCR 초기값 설정**: TPM PCR 10번의 초기값 또는 이전 상태값 사용
2. **로그 엔트리 파싱**: 각 측정 로그 라인을 AST 엔트리로 파싱
3. **해시 확장**: 각 엔트리의 템플릿 해시를 PCR 값에 확장
4. **정책 검증**: 런타임 정책에 따라 파일 허용 여부 확인
5. **서명 검증**: IMA 서명이 있는 경우 디지털 서명 검증

8.3.3 검증 모드별 처리

```
ima_validator = ast.Validator({

ast.ImaSig: functools.partial(_validate_ima_sig, exclude_list_compiled_regex,

ima_keyrings, runtime_policy),

ast.ImaNg: functools.partial(_validate_ima_ng, exclude_list_compiled_regex,

runtime_policy),

ast.Ima: functools.partial(_validate_ima_ng, exclude_list_compiled_regex,

runtime_policy),

ast.ImaBuf: functools.partial(_validate_ima_buf, exclude_list_compiled_regex,

runtime_policy, ima_keyrings, dm_validator)

})
```

8.4 keylime/ima/file_signatures.py IMA 키링 관리

8.4.1 ImaKeyring 클래스 구조

```
class ImaKeyring:
"""IMA 키링에서 키들을 keyid로 인덱싱하여 관리"""
ringv2: Dict[int, SupportedKeyTypes] # keyidv2 -> 공개키 매핑
def  add_pubkey(self, pubkey: SupportedKeyTypes, keyidv2: Optional[int]) -> None
"""공개키를 키링에 추가"""
def  get_pubkey_by_keyidv2(self, keyidv2: int) -> Optional[SupportedKeyTypes]:
"""keyidv2로 공개키 조회"""
@staticmethod
def  _get_keyidv2(pubkey: SupportedKeyTypes) -> int:
"""공개키의 keyidv2 계산 (SHA1 해시의 하위 4바이트)"""
```

8.4.2 지원되는 키 타입

```
SupportedKeyTypes = Union[RSAPublicKey, EllipticCurvePublicKey]


# 지원되는 해시 알고리즘
HASH_FUNCS: Dict[int, hashes.HashAlgorithm] = {
HashAlgo.HASH_ALGO_MD5: hashes.MD5(),
HashAlgo.HASH_ALGO_SHA1: hashes.SHA1(),
HashAlgo.HASH_ALGO_SHA256: hashes.SHA256(),
HashAlgo.HASH_ALGO_SHA384: hashes.SHA384(),
```

```
HashAlgo.HASH_ALGO_SHA512: hashes.SHA512(),
HashAlgo.HASH_ALGO_SHA224: hashes.SHA224(),
# Streebog 알고리즘도 지원
}
```

8.4.3 ImaKeyrings 다중 키링 관리

```
class ImaKeyrings:
"""시스템의 여러 IMA 키링들을 관리"""
keyrings: Dict[str, ImaKeyring] # 키링 이름 -> ImaKeyring 매핑
def add_pubkey_to_keyring(self, pubkey: SupportedKeyTypes,
keyring_name: str, keyidv2: Optional[int] = None) -> None:
"""지정된 키링에 공개키 추가"""
def set_tenant_keyring(self, tenant_keyring: Optional[ImaKeyring]) -> None:
"""테넌트 키링 설정 (명령줄에서 제공된 키들)"""
def integrity_digsig_verify(self, signature: bytes, filehash: bytes,
filehash_type: str) -> bool:
"""모든 키링을 사용하여 서명 검증"""
```

8.5 파일 서명 검증 프로세스

8.5.1 서명 검증 단계

1. **서명 헤더 파싱**: `struct signature_v2_hdr` 형식으로 서명 데이터 파싱

2. **키 조회**: keyidv2를 사용하여 적절한 공개키 찾기

3. **해시 알고리즘 확인**: 서명에 명시된 해시 알고리즘과 파일 해시 타입 일치 확인

4. **서명 검증**: RSA/ECC 알고리즘에 따라 디지털 서명 검증

8.5.2 서명 검증 함수

```
def _asymmetric_verify(self, signature: bytes, filehash: bytes,
filehash_type: str) -> bool:
"""IMA 서명 검증 (커널의 digsig_asymmetric.c 구현과 유사)"""
# 서명 헤더 파싱 (빅엔디안)
fmt = ">BBBIH"
_, _, hash_algo, keyidv2, sig_size = struct.unpack(fmt, signature[:hdrlen])
# 해시 함수 확인
hashfunc = HASH_FUNCS.get(hash_algo)
if not hashfunc or filehash_type != hashfunc.name:
return False
# 키 조회
pubkey = None
for keyring in self.get_all_keyrings():
pubkey = keyring.get_pubkey_by_keyidv2(keyidv2)
if pubkey:
break
# 서명 검증
if pubkey:
```

```
try:
ImaKeyrings._verify(pubkey, signature[hdrlen:], filehash, hashfunc)
return True
except InvalidSignature:
return False
return False
```

8.5.3 키 형식 지원

`get_pubkey()` 함수는 다양한 키 형식을 지원합니다:

- **DER/PEM X.509 인증서**: Subject Key Identifier에서 keyidv2 추출
- **DER/PEM 공개키**: 직접 공개키 로드
- **DER/PEM 개인키**: 개인키에서 공개키 추출

8.6 IMA 정책 검증 및 오류 처리

8.6.1 정책 검증 함수들

```
def  _validate_ima_ng(exclude_regex, runtime_policy, digest, path, hash_types="d
"""IMA-NG 형식 검증"""
if runtime_policy is not None:
# 제외 목록 확인
if exclude_regex and exclude_regex.match(path.name):
return failure
```

```
# 허용 목록에서 파일 확인

accept_list = runtime_policy[hash_types].get(path.name, None)

if accept_list is None:

failure.add_event("not_in_allowlist", f"File not found in allowlist: {path.name}

return failure

# 해시값 확인

hex_hash = digest.hash.hex()

if hex_hash not in accept_list:

failure.add_event("runtime_policy_hash", {

"message": "Hash not found in runtime policy",

"got": hex_hash,

"expected": accept_list

})


def  _validate_ima_sig(exclude_regex, ima_keyrings, runtime_policy, digest, path

"""IMA-SIG 형식 검증 (서명 우선, 실패시 정책 확인)"""

if ima_keyrings and signature:

if ima_keyrings.integrity_digsig_verify(signature.data, digest.hash, digest.algo

return failure # 서명 검증 성공

# 서명 검증 실패시 런타임 정책으로 대체 검증

if runtime_policy is not None:

return _validate_ima_ng(exclude_regex, runtime_policy, digest, path)
```

8.6.2 오류 유형 및 처리

1. **파싱 오류**: 잘못된 IMA 로그 엔트리 형식
2. **PCR 불일치**: 계산된 PCR 값과 TPM PCR 값 불일치
3. **정책 위반**: 허용되지 않은 파일 실행
4. **서명 검증 실패**: 잘못된 디지털 서명
5. **ToMToU 오류**: Time-of-Measure, Time-of-Use 오류 (FF 해시)

8.6.3 증분 증명 지원

```
if agentAttestState is not None:
# 증분 증명을 위한 상태 업데이트
agentAttestState.update_ima_attestation(int(entry.pcr), running_hash, linenum +
if dm_validator:
agentAttestState.set_ima_dm_state(dm_validator.state_dump())
```

8.7 디바이스 매퍼 통합

8.7.1 DM-IMA 지원

Keylime은 Device Mapper IMA 이벤트도 지원합니다:

- **dm_table_load**: 디바이스 매퍼 테이블 로드
- **dm_device_resume**: 디바이스 재개
- **dm_device_remove**: 디바이스 제거
- **dm_table_clear**: 테이블 클리어

- **dm_device_rename**: 디바이스 이름 변경
- **dm_target_update**: 타겟 업데이트

8.7.2 DM 정책 검증

```
def _validate_ima_buf(exclude_regex, runtime_policy, ima_keyrings, dm_validator
digest, path, data):
"""IMA-BUF 형식 검증 (키 또는 DM 이벤트)"""
# 공개키인지 확인
try:
pubkey, keyidv2 = file_signatures.get_pubkey(data.data)
if pubkey:
# 키링에 키 추가 (정책 검증 후)
failure = _validate_ima_ng(exclude_regex, runtime_policy, digest, path, "keyring
if not failure and ima_keyrings is not None:
ima_keyrings.add_pubkey_to_keyring(pubkey, path.name, keyidv2=keyidv2)
except ValueError:
# DM 이벤트인지 확인
if dm_validator is not None and path.name in dm_validator.valid_names:
failure = dm_validator.validate(digest, path, data)
else:
# 일반적인 ima-buf 엔트리
failure = _validate_ima_ng(exclude_regex, runtime_policy, digest, path, "ima-buf
```

이 IMA 통합 분석을 통해 Keylime이 어떻게 Linux IMA 시스템과 통합되어 실시간 파일 무결성 검증을 수행하는지 이해할 수 있습니다. 런타임 정책, 키링 관리, 서명 검증 등의 복합적인 메커니즘을 통해 강력한 무결성 보장을 제공합니다.